OPERATING SYSTEMS

Chapter 5: Threads

Prepared by: yasmeen Alshraideh

ROOT TEAM
ROOT : WHERE SECURITY BEGINS

Slide 2 - Threads

- A thread, sometimes called a lightweight process (LWP), is a basic unit of CPU utilization.
- It comprises a thread ID, a program counter, a register set, and a stack.
- It shares with other threads belonging to the same process its code section, data section, and other OS resources, such as open files.
- A traditional (or heavyweight) process has a single thread of control.
- If the process has multiple threads of control, it can do more than one task at a time.

الـ Threadهو أصغر وحدة تنفيذ (Unit of Execution) داخل الـProcess ، ويُسمّى أحيانًا:

Lightweight Process (LWP)

يعني بدل ما نفكر بالـ Process على أنه كتلة واحدة تنفّذ كل شيء، ممكن يكون داخله عدةThreads ، وكل Thread مسؤول عن تنفيذ مسار معين من التعليمات .

كل Thread يمتلك معلومات خاصة فيه:

Thread ID: ر قم أو معرّف يميز الـ Thread عن غيره.

Program Counter: يحتفظ بعنوان التعليمة التالية التي سيتم تنفيذها.

Register Set: مجموعة Registers تحتوي على القيم اللازمة أثناء التنفيذ.

Stack: مساحه خاصه بال Thread تُستخدم لمعلومات استدعاء الدوال والمتغيرات المحلية وغيرها من بيانات التنفيذ

Threadsالتابعة لنفس الـ Process تشترك في:

- Code Section: تعليمات البرنامج.
- Data Section: بيانات البرنامج المشتركة.
- OS Resources: مثل الملفات المفتوحة.(Open Files)

يعني إذا كان عنا Process يحتوي على ثلاثة Threads، فهم يقدروا يتعاملوا مع نفس كود البرنامج وبياناته وموارده.

لكن كل Thread يحتفظ بمعلومات التنفيذ الخاصة فيه، مثل Program Counter و Registersو Stack.

Single-threaded Process: الـ Process يحتوي على Thread واحد فقط، وبالتالي عنده مسار تنفيذ واحد.

Multithreaded Process: الـ Process يحتوي على أكثر منThread ، وبالتالي يمكنه تنفيذ أكثر من مهمة، وقد تعمل المهام بالتزامن أو بالتوازي حسب المعمارية والجدولة.

Slide 3 - Threads (Examples)

- Many software packages that run on modern desktop PCs are multithreaded.
- An application typically is implemented as a separate process with several threads of control.
- Examples:
  - A web browser might have one thread display images or text while another thread retrieves data from the network.
  - A word processor may have a thread for displaying graphics, another for reading keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

- البرامج الحديثة تستخدمMultithreading ، أي أنها تحتوي على عدة Threads داخل نفس التطبيق.
- عادةً يتم تنفيذ التطبيق كـ Process مستقل، وداخله عدة مسارات تنفيذ.

Slide 4 - Multithreaded Process

- Address space + Threads + Resources
  - Address space: contains code and data of a process.
  - Threads: are individual execution contexts.
  - Resources: are physical support necessary to run the process (memory, disk, …).

الـ Multithreaded Process يتكون من ثلاثة عناصر رئيسية:

1. Address Space

هي مساحة العناوين الخاصة بالـProcess ، وتحتوي على:

- Code
- Data

أي أنها تمثل المساحة التي توجد فيها تعليمات البرنامج وبياناته.

2. Threads

هي مسارات التنفيذ المستقلة داخل الـProcess.

كل Thread يمتلك Execution Context خاصًا به، أي المعلومات التي يحتاجها لمتابعة التنفيذ.

3. Resources

هي الموارد اللازمة لتشغيل الـProcess ، مثل:

- Memory
- Disk

Slide 5 - Single and Multithreaded Processes

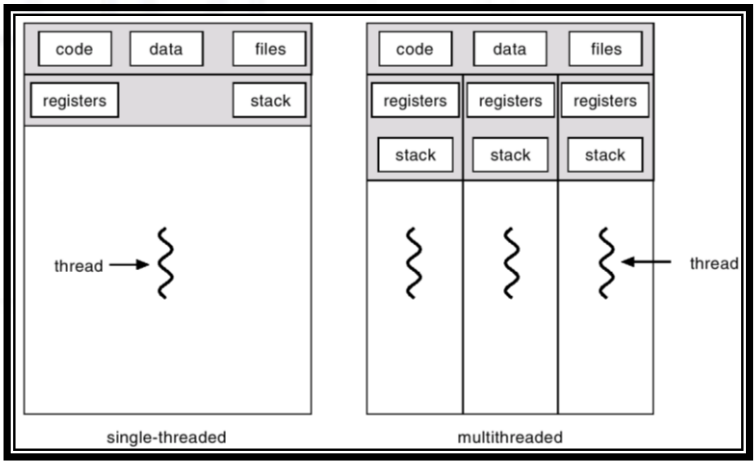
الرسم يقارن بين Process يحتوي على Thread واحد وآخر يحتوي على عدة Threads.

مشترك بين ال threads:

- Code
- Data
- Files

خاص لكل thread :

- Thread ID
- Program Counter
- Registers
- Stack



Slide 6+7 - Benefits of Multithreading

1. Responsiveness

Example: A multithreaded web browser could still allow user interaction in one thread while an image is being loaded in another thread.

2. Resource Sharing
   - By default, threads share the memory and the resources of the process to which they belong.
   - The benefit: it allows an application to have several different threads of activity all within the same address space.
3. Economy
   - Allocating memory and resources for process creation is costly.
   - It is more economical to create and context switch threads because they share resources.
   - In general, it is much more time consuming to create and manage processes than threads.
4. Utilization of Multiprocessor Architectures
   - Each thread may be running in parallel on a different processor.
   - A single-threaded process can only run on one CPU, no matter how many are available.
   - Multithreading on a multi-CPU machine increases concurrency.

اربع فوائد لاستخدام Multithreading :

1. Responsiveness - الاستجابة

- تعني أن التطبيق يظل قادرًا على الاستجابة لتفاعل المستخدم حتى لو كان أحد الـ Threads مشغولًا بتنفيذ مهمة أخرى.
- مثال:Web Browser
  - Thread 1: تحميل صورة.
  - Thread 2: التعامل مع المستخدم.
- إذا كان Thread التحميل مشغولًا، يمكن أن يستمر Thread آخر في التعامل مع المستخدم.

بالتالي، لا يحتاج التطبيق إلى التوقف عن الاستجابة بالكامل بسبب مهمة واحدة.

2. Resource Sharing - مشاركة الموارد

- الـ Threads الموجودة داخل نفس الـ Process تشترك افتراضيًا في الذاكرة والموارد التابعة لذلك الـProcess .

وهذا يسمح للتطبيق بتنفيذ عدة أنشطة ضمن نفسAddress Space .

3. Economy - الاقتصاد في الموارد والوقت

- إنشاء Process جديد يحتاج تخصيص Memory و Resourcesجديدة، ما قد يكون مكلف من ناحية الوقت والموارد.
- أما إنشاء Thread داخل Process موجود، فيكون أكثر اقتصادية لأنه يشارك الموارد الموجودة أصلًا.
- كذلك، عمليات Context Switching بين Threads تكون أقل تكلفة بشكل عام من عمليات إنشاء وإدارةProcesses .
- Context Switch: عملية الانتقال من تنفيذ Thread أو Process إلى آخر، مع حفظ واستعادة معلومات التنفيذ اللازمة.

4.Utilization of Multiprocessor Architectures

- تعني الاستفادة من الأجهزة التي تحتوي على أكثر من CPU أو أكثر منCore .
- إذا كان لدينا Multithreaded Process ، يمكن جدولة Threads مختلفة على معالجات مختلفة، مما يسمح بالتنفيذ المتوازي.
- أما Single-threaded Process ، فلا يمكنه تشغيل أكثر من Thread واحد في نفس اللحظة، حتى لو كان الجهاز يحتوي على عدةCPUs .

Slide 8 - Single vs. Multiprocessor

Single-processor architecture:

Multithreading generally occurs by time-division multiplexing (as in multitasking).

The CPU generally moves between each thread so quickly as to create an illusion of parallelism, but in reality only one thread is running at all time.

Multiprocessor or multi-core system:

The threads or tasks will generally run at the same time, with each processor or core running a particular thread or task.

الفرق بين طريقة تنفيذ Threads على جهاز يحتوي على معالج واحد وجهاز يحتوي على عدة معالجات أو Cores.

1. Single-Processor Architecture

في نظام يحتوي على CPU واحد، يتم التبديل بين Threads باستخدام:

Time-Division Multiplexing

أي أن المعالج يعطي كل Thread فترة زمنية قصيرة، ثم ينتقل إلى Thread آخر.

لأن الانتقال سريع جدًا، يشعر المستخدم وكأن عدة Threads تعمل بالتوازي، لكن فعليًا في كل لحظة ينفّذ المعالج Thread واحدًا فقط.

هذا يُسمّى: Concurrency without Parallelism

2. Multiprocessor / Multi-core System

إذا كان الجهاز يحتوي على أكثر من Processor أو Core ، يمكن تنفيذ Threads مختلفة في الوقت نفسه.

كل Processor أو Core يستطيع تشغيل Thread أو Task معين.

هذا يُسمّى: Parallelism

Slide 9 - User and Kernel Threads

Support for threads may be provided at either:

- The user level, for user threads.
- By the kernel, for kernel threads.

يوجد مستويان لإدارة Threads:

1. User-Level Threads: تتم إدارتها على مستوى User Space باستخدام Thread Library.
2. Kernel Threads: تتم إدارتها بواسطة Kernel الخاص بنظام التشغيل.

الفرق الأساسي بينهما هو الجهة المسؤولة عن إنشاء Threads وجدولتها وإدارتها.

Slide 10 - User Threads

Thread management done by user-level threads library

- Supported and managed above the kernel and are implemented by a thread library at the user level.
- The library provides support for thread creation, scheduling, and management with no support from the kernel.
- The kernel is unaware of user-level threads; all thread creation and scheduling are done in user space without the need for kernel intervention.
- User-level threads are generally fast to create and manage

الUser-Level Threads هي Threads يتم إنشاؤها وإدارتها بواسطة مكتبة تسمى: User-Level Thread Library

وتعمل هذه المكتبة فيUser Space ، أي فوق مستوى الـKernel.

المكتبة مسؤولة عن:

- Thread Creation
- Thread Scheduling
- Thread Management

والـ Kernel لا يكون على علم بالـ User Threads حيث إنشاء هذه الـ Threads وجدولتها يتمان دون تدخل مباشر من الKernel .

- إنشاء وإدارة User Threads يكونان سريعَين بشكل عام، لأن العمليات تتم في User Space ولا تتطلب تدخل Kernel لكل عملية إدارة.Thread
- الـ Kernel قد يتعامل مع الـ Process باعتباره وحدة التنفيذ المعروفة لديه، دون أن يعرف التفاصيل الداخلية للـ User Threads التابعة له.

Slide 11 - User Threads (Cont.)

Drawback: If the kernel is single-threaded, then any user-level thread performing a blocking system call will cause the entire process to block, even if other threads are available to run within the application.

Examples:

- POSIX Pthreads
- Mach C-threads
- Solaris 2 UI-threads

أهم عيب لل User Thread هو أن تنفيذ Blocking System Call بواسطته قد يؤدي إلى توقف الـ Process بالكامل، إذا كان الـ Kernel لا يدعم تعدد Threads بالطريقة اللازمة.

الBlocking System Call هو System Call قد يجعل العملية تنتظر إلى أن تنتهي عملية معينة أو يصبح مورد معين متاحًا.

كيف تحدث المشكلة؟

لنفترض أن لدينا Process يحتوي على ثلاثة:User Threads

إذا نفذ Thread A عملية Blocking System Call ، فقد يتوقف الـ Process كله، وبالتالي لا تستطيع Threads الأخرى الاستمرار بالتنفيذ، حتى لو كانت جاهزة للعمل.

امثله:

1. POSIX Pthreads
2. Mach C-threads
3. Solaris 2 UI-threads

Slide 12 - Kernel Threads

Supported and managed by the Kernel (OS).

- The kernel performs thread creation, scheduling, and management in kernel space.
- Kernel threads are generally slower to create and manage than user threads.
- If a thread performs a blocking system call, the kernel can schedule another thread in the application for execution.
- In a multiprocessor environment, the kernel can schedule threads on different processors.

Examples:

- Windows 95/98/NT/2000
- Solaris 2
- Tru64 UNIX
- BeOS

الـ**Kernel Threads** هي Threads تتم إدارتها مباشرة بواسطة Kernel الخاص بنظام التشغيل.

الـ Kernel مسؤول عن:

- Thread Creation
- Thread Scheduling
- Thread Management

وتتم هذه العمليات في.Kernel Space

الايجابيات:

1.Handling Blocking System Calls

إذا نفذ Thread عمليةBlocking System Call ، يستطيع Kernel جدولة Thread آخر من نفس التطبيق للتنفيذ.

يعني مش بالضرورة يتوقف التطبيق بالكامل بسبب توقف Thread واحد.

2.Multiprocessor Support

في الأنظمة التي تحتوي على أكثر منProcessor ، يستطيع Kernel جدولة Threads مختلفة على معالجات مختلفة.

السلبيات:

إنشاء وإدارة Kernel Threads أبطأ بشكل عام منUser Threads ، بسبب الحاجة إلى تدخل.Kernel

Examples

- Windows 95/98/NT/2000
- Solaris 2
- Tru64 UNIX
- BeOS

Slide 13 - Multithreading Models

The three multithreading models are:

1. Many-to-One (User-level threading)
2. One-to-One (Kernel-level threading)
3. Many-to-Many (Hybrid threading)

الـ Multithreading Models توضح كيفية ربط User-Level Threads معKernel Threads.

يوجد ثلاثة نماذج رئيسية:

1. Many-to-One: Many User Threads **TO** One Kernel Thread
2. One-to-One: One User Thread **TO** One Kernel Thread
3. Many-to-Many: Many User Threads **TO** Multiple Kernel Threads

الاختلاف الأساسي بين النماذج هو عدد الـ User Threads التي يمكن ربطها بكلKernel Thread ، وما يترتب على ذلك من تأثير على Blockingو Parallelismو Overhead.

Slide 14+15 - Many-to-One Model

- Many user-level threads mapped to single kernel thread.
- It is efficient because thread management is done in user space.
- Shortcoming: The entire process will block if a thread makes a blocking system call.
- Only one thread can access the kernel at a time; therefore, multiple threads are unable to run in parallel on multiprocessors.
- Used on systems that do not support kernel threads.

في نموذج Many-to-One ، يتم ربط عدة User Threads مع Kernel Thread واحد فقط

Advantages :Efficient Thread Management

إدارة الـ Threads تتم فيUser Space ، لذلك تكون عملية الإدارة فعّالة من ناحية التكلفة.

Disadvantages:

1.Blocking Problem

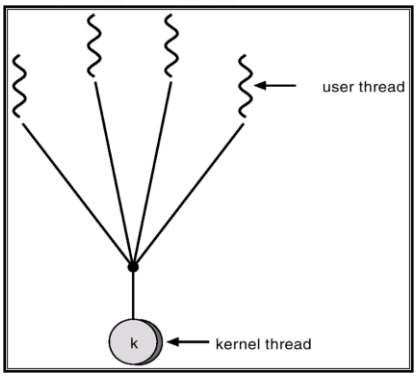
إذا نفّذ أحد الـ User Threads عمليةBlocking System Call ، قد يتوقف الـ Process بالكامل.

2.No Parallelism on Multiprocessors

لأن جميع User Threads مرتبطة بـ Kernel Thread واحد، لا يمكن تشغيلها بالتوازي على عدة معالجات.

Usage: يُستخدم في الأنظمة التي لا تدعمKernel Threads .

*مهما كان عدد User Threads ، فإنها تعتمد على Kernel Thread واحد فقط.



Slide 16+17 - One-to-One Model

- Each user-level thread maps to kernel thread.
- Allows more concurrency than many-to-one by allowing another thread to run when a thread makes a blocking system call.
- It also allows multiple threads to run in parallel on multiprocessors.
- Shortcoming: The overhead of creating kernel threads can burden the performance of an application.

Examples:

- Windows 95/98/NT/2000
- OS/2

في نموذجOne-to-One ، كل User Thread يتم ربطه مع Kernel Thread مستقل.

Advantages

More Concurrency.1

إذا نفذ Thread عملية Blocking System Call ، يستطيع Kernel جدولة Thread آخر للتنفيذ.

Parallel Execution.2

يمكن تشغيل عدة Threads بالتوازي على Multiprocessors ، لأن كل User Thread لديه Kernel Thread مقابل.

Disadvantage

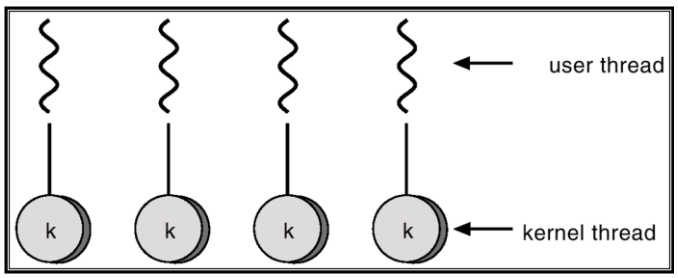
Kernel Thread Creation Overhead

إنشاء Kernel Threads يحتاج موارد وعمليات إدارة منKernel ، وهذا قد يؤثر على أداء التطبيق إذا كان عدد الـ Threads كبيرًا.

Examples

- Windows 95/98/NT/2000
- OS/2

* كل User Thread له Kernel Thread مقابل، وهذا يسمح للـ Kernelبجدولة Threads بشكل مستقل.



Slide 18+19+20 - Many-to-Many Model (N : M)

- Multiplexes many user-level threads to a smaller or equal number of kernel threads.
- Allows the operating system to create a sufficient number of kernel threads.
- Suffers from neither of the shortcomings of the other two approaches (1:1 and 1:N).
- Developers can create as many user threads as necessary.
- The corresponding kernel threads can run in parallel on a multiprocessor.
- The kernel can schedule another thread for execution when a thread performs a blocking system call.

Examples:

- Solaris 2
- IRIX
- HP-UX
- Tru64 UNIX

في نموذجMany-to-Many ، يتم ربط عدد كبير من User Threads بعدد أصغر أو مساوٍ منKernel Threads.

Main Features

1.Flexible Thread Creation

يمكن للمطور إنشاء العدد الذي يحتاجه منUser Threads.

2.Sufficient Kernel Threads

نظام التشغيل يستطيع إنشاء عدد مناسب من Kernel Threads لإدارة التنفيذ.

3.Parallelism

يمكن تشغيل Kernel Threads بالتوازي علىMultiprocessor System.

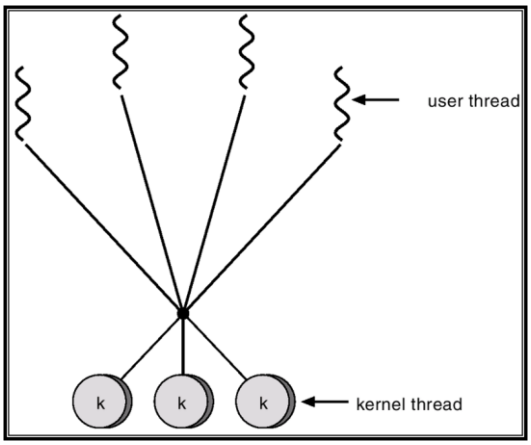
4.Blocking System Calls

إذا نفّذ Thread عمليةBlocking System Call ، يستطيع Kernel جدولة Thread آخر للتنفيذ.

هذا النموذج يتجنب أوجه القصور المذكورة في النموذجين الآخرين، أي:

- مشكلة توقف كامل الـ Process المرتبطة بـMany-to-One.
- عبء إنشاء Kernel Thread مستقل لكل User Thread فيOne-to-One.
- أمثلة على أنظمة وردت ضمنMany-to-Many Model:

1. Solaris 2
2. IRIX
3. HP-UX
4. Tru64 UNIX



* عدة User Threads يمكن أن تُجدول على عدد أقل منKernel Threads ، وأن Kernel Threads يمكن أن تعمل على معالجات متعددة.

هذا النموذج يجمع بين مرونة إنشاء User Threads وإمكانية الاستفادة من Kernel Schedulingو Parallelism.

Chapter 5 – Comprehensive Review

Important Definitions

Term	Definition / Key Idea
Thread (Lightweight Process, LWP)	A basic unit of CPU utilization: a thread ID, a program counter, a register set, and a stack.
Heavyweight Process	A traditional process that has a single thread of control.
Multithreaded Process	A process with several threads of control that can do more than one task at a time.
Shared by Threads of a Process	Code section, data section, and other OS resources such as open files.
Responsiveness	A multithreaded application can keep interacting with the user while another thread works (e.g., loading an image).
Resource Sharing	Threads share the memory and resources of their process by default.
Economy	Creating and context switching threads is cheaper than creating and managing processes.
Utilization of Multiprocessor Architectures	Threads can run in parallel on different processors, increasing concurrency.
User Threads	Supported and managed above the kernel by a user-level thread library, without kernel support.
Kernel Threads	Supported and managed by the kernel; creation, scheduling, and management are done in kernel space.
Blocking System Call	A system call that blocks the caller; with user threads on a single-threaded kernel it blocks the entire process.
Many-to-One	Many user-level threads mapped to a single kernel thread.
One-to-One	Each user-level thread maps to its own kernel thread.
Many-to-Many (N:M)	Multiplexes many user-level threads to a smaller or equal number of kernel threads.
Time-Division Multiplexing	On a single processor, the CPU moves between threads so quickly that it creates an illusion of parallelism.

Benefits of Multithreading

Benefit	Meaning
Responsiveness	The application keeps responding to the user while another thread is busy.
Resource Sharing	Threads share the memory and resources of their process, all in the same address space.
Economy	Cheaper to create and context switch than processes, since resources are shared.
Utilization of Multiprocessor Architectures	Each thread may run in parallel on a different processor; increases concurrency.

Comparison: Process vs. Thread

Heavyweight Process	Thread (Lightweight Process)
A traditional process with a single thread of control.	A basic unit of CPU utilization inside a process.
Allocating memory and resources for creation is costly.	More economical to create and context switch, because threads share resources.
Much more time consuming to create and manage.	Less time consuming to create and manage.
Has its own address space and resources.	Shares code, data, and OS resources with the other threads of the same process.
Single-threaded: runs on only one CPU.	Each thread may run in parallel on a different processor.

Comparison: Single-Threaded vs. Multithreaded Process

Single-Threaded Process	Multithreaded Process
One thread of control.	Several threads of control.
One set of registers and one stack.	A separate register set and stack for each thread.
Does one task at a time.	Can do more than one task at a time.
Can only run on one CPU, no matter how many are available.	On a multi-CPU machine, increases concurrency.

Comparison: Single-Processor vs. Multiprocessor Multithreading

Single-Processor	Multiprocessor / Multi-Core
Time-division multiplexing (as in multitasking).	Threads generally run at the same time.
The CPU moves between threads so quickly that it creates an illusion of parallelism.	Each processor or core runs a particular thread or task.
Only one thread is running at all times.	True parallel execution of several threads.

Comparison: User Threads vs. Kernel Threads

User Threads	Kernel Threads
Managed by a user-level threads library.	Supported and managed by the kernel (OS).
Creation, scheduling, and management in user space.	Creation, scheduling, and management in kernel space.
The kernel is unaware of them.	The kernel knows about them and schedules them.
Generally fast to create and manage.	Generally slower to create and manage.
A blocking system call can block the entire process (single-threaded kernel).	If one thread blocks, the kernel can schedule another thread of the application.
Examples: POSIX Pthreads, Mach C-threads, Solaris 2 UI-threads.	In a multiprocessor environment, the kernel can schedule threads on different processors. Examples: Windows 95/98/NT/2000, Solaris 2, Tru64 UNIX, BeOS.

Comparison: Many-to-One vs. One-to-One

Many-to-One	One-to-One
Many user threads mapped to a single kernel thread.	Each user thread maps to its own kernel thread.
Efficient: thread management is in user space.	More concurrency than many-to-one.
The entire process blocks if a thread makes a blocking system call.	Another thread can run when a thread makes a blocking system call.
Multiple threads cannot run in parallel on multiprocessors.	Multiple threads can run in parallel on multiprocessors.
Used on systems without kernel-thread support.	Shortcoming: the overhead of creating kernel threads can burden performance.

Comparison: One-to-One vs. Many-to-Many

One-to-One	Many-to-Many (N:M)
Number of kernel threads equals the number of user threads.	Kernel threads are fewer than or equal to user threads.
Creating many kernel threads burdens performance.	The OS can create a sufficient number of kernel threads; no such shortcoming.
Each user thread is paired with a kernel thread of its own.	Developers can create as many user threads as necessary.
Examples: Windows 95/98/NT/2000, OS/2.	Examples: Solaris 2, IRIX, HP-UX, Tru64 UNIX.

RedTeam
ROOT: WHERE SECURITY BEGINS